

System Design Question: Design WhatsApp

#108: System Design Interview - Part 2



NEO KIM AND HAYK

DEC 16, 2025  PAID

Get my system design playbook for FREE on
newsletter signup:

✓ Subscribed

- *Share this post & I'll send you some rewards for the referrals.*
- *Block diagrams created using Eraser.*

Designing a real-time chat app like WhatsApp is a popular system design interview question.

In a short period, you need to show you understand not just the happy path, but also all the things that can go wrong when messages fly between disconnected users on mobile networks.

In Part 1, we covered:

- Core architecture and data models
- Service discovery and presence management
- Handling online and offline message delivery

- And many more!

Today we'll cover:

- Scaling from one server to thousands with Redis Pub/Sub
- Database sharding and replication strategies
- Message storage service internals
- Scheduled jobs and cleanup
- Message deduplication, failure recovery, and edge cases
- End-to-end encryption¹ and key management
- Rate limiting and abuse prevention
- Analytics and monitoring infrastructure
- Multi-region deployment and global distribution
- Schema versioning and backward compatibility

By the end, you'll understand not just what to build, but why each decision is made and what breaks when you get it wrong.

Onward.

Software engineers, are you prepared if layoffs happen again? (Sponsor)



Build a GitHub portfolio with 4 production-grade projects.

- **Build real AI systems:** autonomous agents, recommendation engines, and multimodal pipelines.
- **Master the modern stack:** LangChain, TensorFlow, PyTorch, multi-agent architectures, and RAG systems.

Flexible & Practical:

- 5-month, fully online, project-based program for working engineers
- Live hands-on workshops + 24/7 support

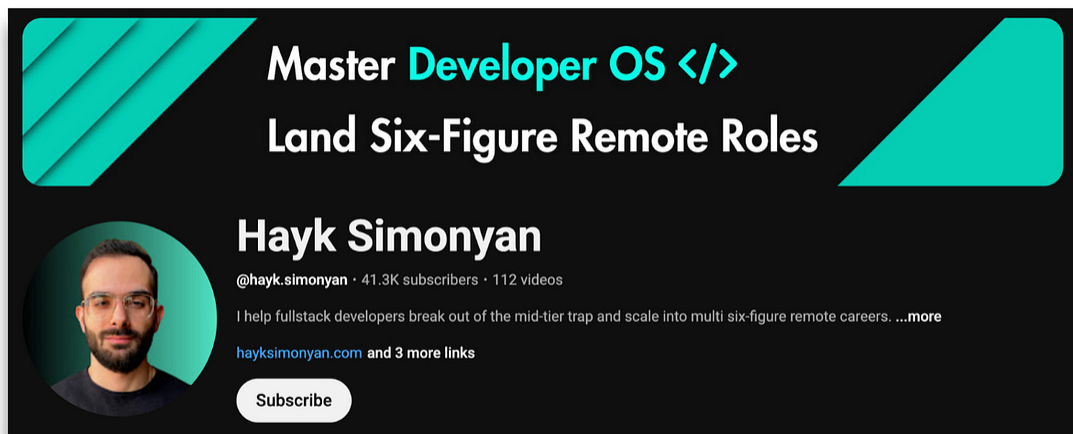
Earn up to 56% more by mastering production-grade AI.

Next cohort starts on 19 January 2026.

Imagine leading AI initiatives with confidence:

[CLICK HERE TO KNOW MORE](#)

I want to reintroduce [Hayk Simonyan](#) as a guest author.

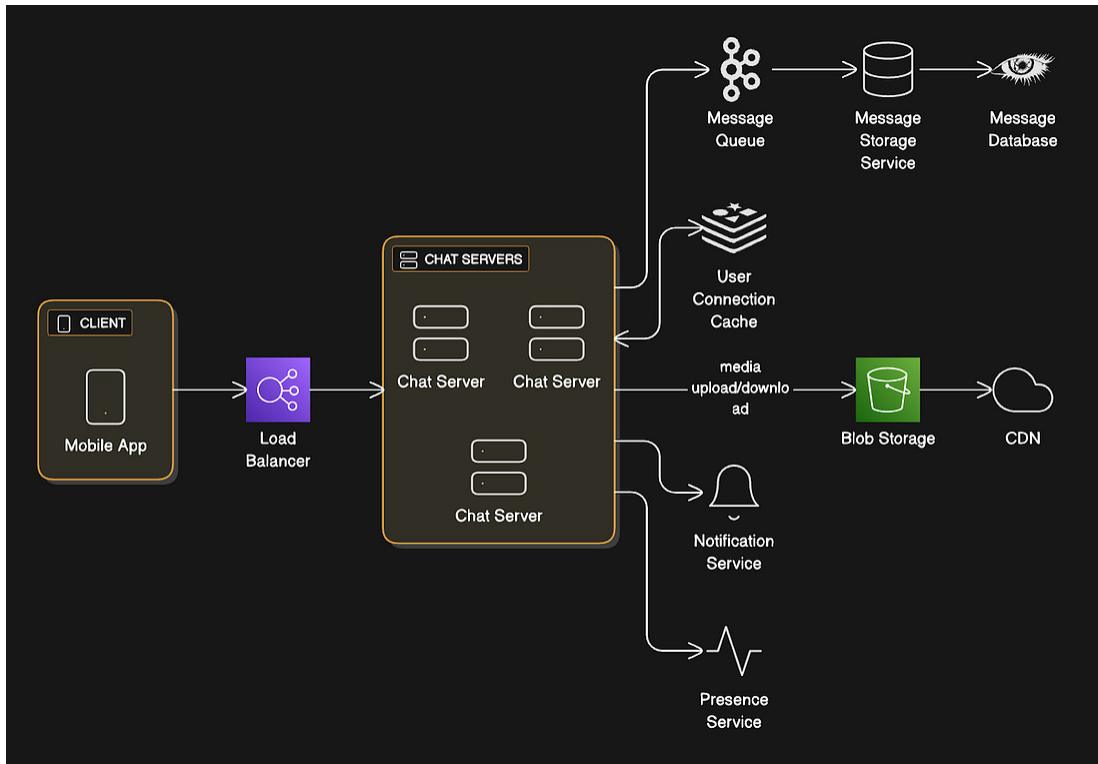


He's a senior software engineer specializing in helping developers break through their career plateaus and secure senior roles.

If you want to master the essential system design skills and land senior developer roles, I highly recommend checking out Hayk's **YouTube channel**.

His approach focuses on what top employers actually care about: system design expertise, advanced project experience, and elite-level interview performance.

Scaling



Our single chat server design works great for 1000 users. But it's terrible for a billion.

So let's scale horizontally...

Problem: Users on Different Servers

User A connects to Server 1. User B connects to Server 2.

So how does a message from User A reach B?

Option 1: Load balancer does nothing

This approach doesn't work. Server 1 receives the message but has no connection to User B.

Option 2: Kafka topics per user

Create a Kafka topic (a category or feed name to which messages are published) for each user. Servers subscribe to topics for their

connected users.

Why this approach won't work?

Kafka needs about 50KB of overhead per topic.

- 1 billion users = 50 TB just for topic metadata.

Kafka wasn't built for billions of topics. It's designed for thousands of high-throughput topics, not billions of low-throughput ones.

Option 3: Consistent hashing

Hash user IDs to specific servers².

Always route the same user to the same server. When a message arrives for User B, forward it directly to Server 2.

Pros:

- Direct server-to-server communication,
- No extra infrastructure.

Cons:

- All-to-all connections between servers (if you have 1000 servers, each needs connections to 999 others).
- Adding new servers requires careful coordination to avoid dropping messages during the transition.
- Servers need to be big to keep the fleet small.

Option 4: Redis Pub/Sub (what we actually use)

This is the cleanest solution at our scale.

How it works:

Redis Pub/Sub³ is a lightweight messaging system built into Redis. It lets servers subscribe to channels and publish messages to those channels. Think of it like a radio station (channel) where servers can tune in to listen and broadcast messages.

- When User B connects to Chat Server 2, Server 2 subscribes to Redis channel of user:B.
- When Server 1 needs to deliver a message to User B, Server 1 publishes the message to Redis channel user:B.
- Redis routes it to Server 2. Then Server 2 delivers to User B via WebSocket⁴.

Why we chose this:

Redis Pub/Sub uses only a few bytes per channel (versus Kafka's 50KB per topic).

It's "at most once" delivery, meaning if no one's listening when you publish, the message is lost. But that's fine for our use case. We already have durability through the Message Queue⁵ and database. Pub/Sub is just for real-time delivery to online users. If they're offline, they still get messages from their inbox.

The beauty is that chat servers don't need to know about each other. They talk only to Redis. Add or remove chat servers at will. No coordination needed.

Tradeoffs:

Redis Pub/Sub adds a few milliseconds of latency (single-digit, usually under 5ms) because messages bounce through Redis.

But this is negligible compared to network latency.

We also need to run a Redis cluster for high availability, which adds operational complexity. Yet the simplicity of not managing server-to-server connections makes it worth it.

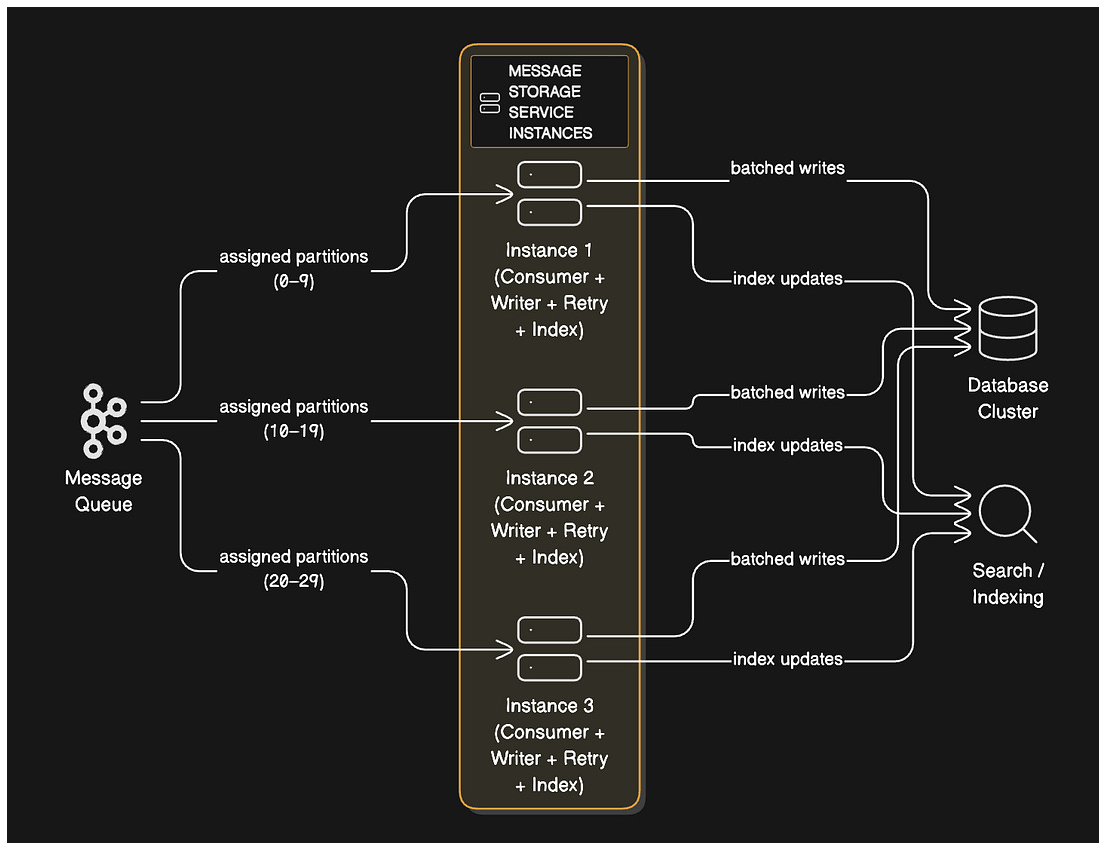
Message Storage Service: The Detailed Flow

The Message Storage Service sits between Message Queue and Database.

Let's break down exactly what it does...

Architecture:

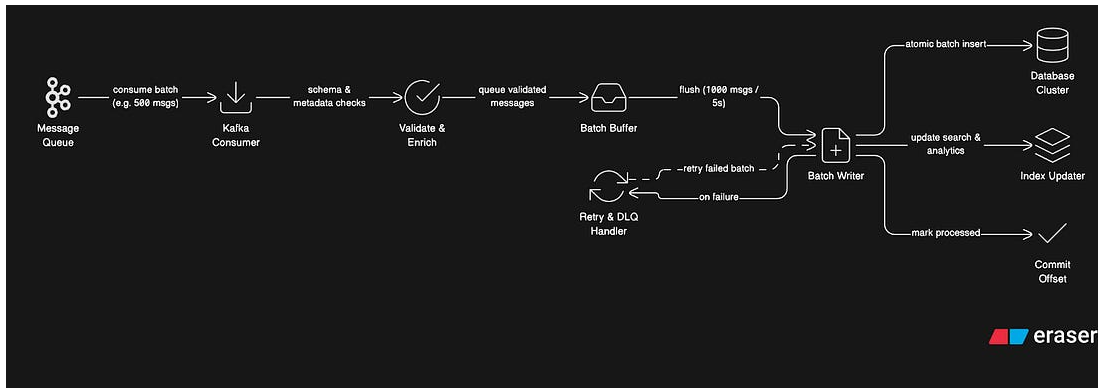
- Message Queue (Kafka)
- Message Storage Service instances
- Database Cluster



Each instance:

- Consumes from assigned Kafka partitions
- Batches writes for efficiency
- Handles retries and failures
- Updates indexes

Detailed Processing Flow:



Step 1: Consuming from Queue

Storage Service Instance:

1. Subscribes to Kafka partitions 0-9 (out of 100 total)
2. Polls for new messages every 100ms
3. Receives a batch of 500 messages
4. Processes batch in parallel (thread pool of 20 workers)

Step 2: Message Validation and Enrichment

For each message:

1. Validate schema (required fields present)
2. Check message size limits (text < 4KB, no huge strings)
3. Enrich with metadata:
 - Calculate message hash (for deduplication)
 - Extract mentions (@username patterns)
 - Identify links for preview generation
 - Classify content type
4. Generate indexes needed for queries

Step 3: Batch Writing

Instead of writing messages one-by-one:

1. Accumulate messages in the memory buffer
2. When buffer hits 1000 messages OR 5 seconds elapsed:
 - Sort messages by target shard
 - Group messages by conversation
 - Prepare batch INSERT statements
3. Write the entire batch in one operation per shard

Why batching matters:

- A single database INSERT takes ~5ms.
- 1000 individual INSERTs = 5 seconds.
- One batched INSERT of 1000 messages = 50ms.

That's a 100x throughput improvement.

Step 4: Writing to Database (Cassandra)

For each shard:



```
Writing to Cassandra

BEGIN BATCH
  INSERT INTO messages (message_id, sender_id, receiver_id, content, ...)
  VALUES ('20241105120000000001', '123', '456', 'Hello', ...);

  INSERT INTO messages (message_id, sender_id, receiver_id, content, ...)
  VALUES ('20241105120000000001', '456', '123', 'Hello', ...);
  // (Same message written to both participants' shards)

  ... (998 more messages)
END BATCH;
```

Cassandra processes this batch atomically. All succeed, or all fail.

Step 5: Updating Indexes

Simultaneously with message write:

1. Update conversation index:
 - Latest message timestamp
 - Unread count
 - Last message preview
2. Update search index (if full-text search enabled):
 - Extract keywords
 - Send to Elasticsearch cluster
3. Update analytics:
 - Increment counters (messages/day, active users)
 - Send to analytics pipeline (separate Kafka topic)

Step 6: Handling Failures

If write fails:

1. Log error with message details
2. Check error type:
 - Temporary (connection timeout): Retry with exponential backoff⁶
 - Permanent (validation error): Move to dead-letter⁷ queue for investigation
 - Duplicate key: Skip (already written, this is retry)
3. Don't block other messages
4. Update monitoring metrics

Step 7: Committing Offset

After a successful write:

1. Commit Kafka offset (mark messages as processed)

2. This ensures we don't reprocess on restart
3. If service crashes before commit:
 - On restart, reprocess from last committed offset
 - Duplicates handled by checking `message_id`

Scaling the Storage Service:

Each instance is stateless. To handle more throughput:

1. Add more instances (they auto-distribute Kafka partitions)
2. Each instance processes independently
3. No coordination needed between instances
4. Kafka handles partition assignment

If you have 10 instances and add an 11th, Kafka automatically rebalances. Some instances give up partitions to the new instance. Within seconds, the load is redistributed.

Monitoring Critical Metrics:

- Consumer lag: How many messages behind are we?
 - (Alert if lag > 10,000 messages)
- Write latency: How long does each batch take?
 - (Alert if p99 > 500ms)
- Error rate: How many failures?
 - (Alert if > 0.1%)
- Throughput: Messages per second processed
 - (Scale up if maxed out)

Tradeoffs:

Message Storage Service adds latency (messages sit in the queue before being written).

But it decouples chat servers from database performance. Chat servers can acknowledge messages immediately without waiting for database writes.

This keeps user-facing latency low even if the database is temporarily slow.

End-to-End Encryption and Key Management

WhatsApp's defining feature is end-to-end encryption.

Messages are encrypted on the sender's device and only decrypted on the recipient's device. Even WhatsApp's servers can't read the content.

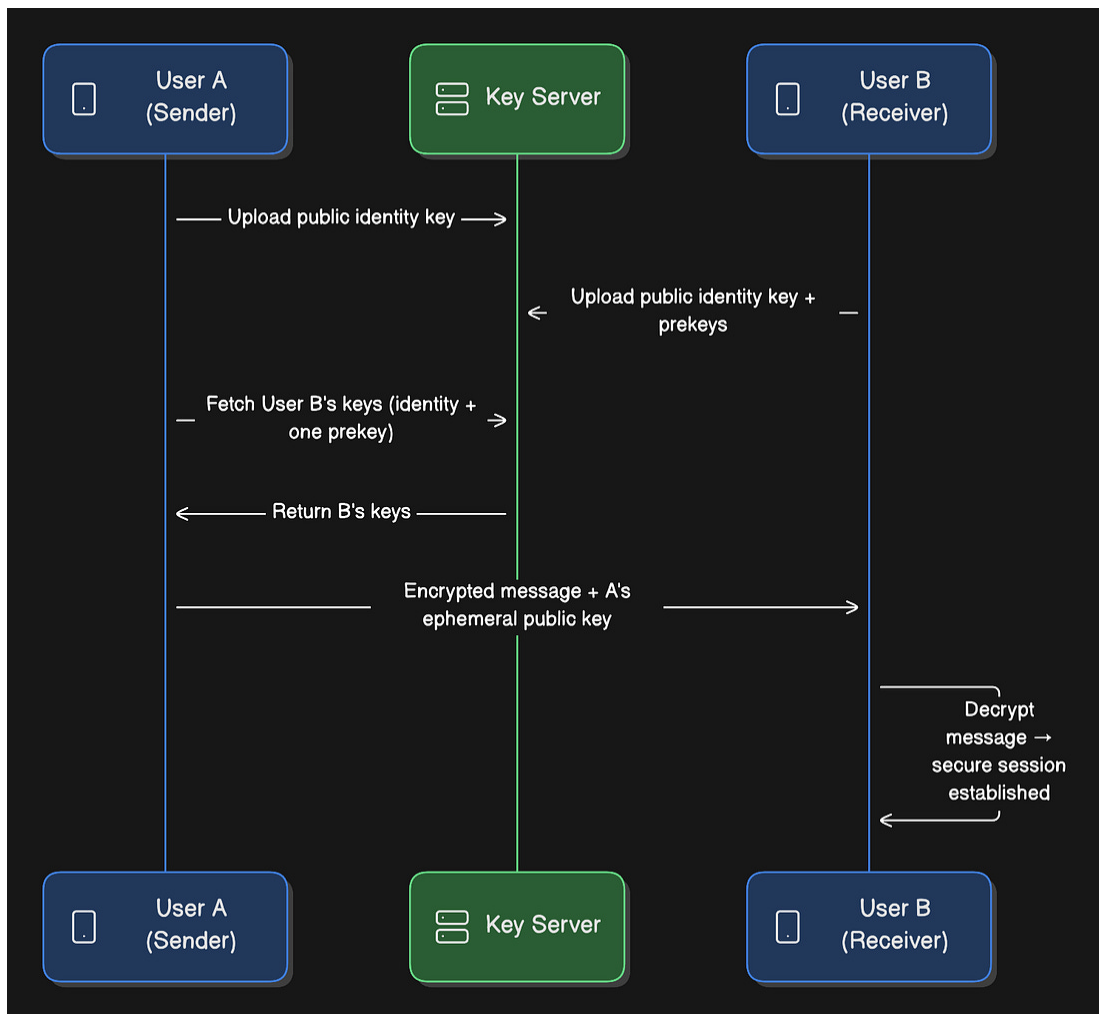
The challenge is how to encrypt messages when users might have multiple devices, be offline, or need to verify identities?

Signal Protocol (What WhatsApp Uses):

WhatsApp implements the Signal Protocol, which provides:

- End-to-end encryption
- Forward secrecy⁸ (compromising one message doesn't compromise others)
- Future secrecy (compromising a key doesn't compromise future messages)

Key Exchange Flow:



When User A first messages User B:

1. Identity Keys (long-term):

- Each user generates an identity key pair on registration
- Public key gets uploaded to server
- Private key never leaves device

2. Prekeys (medium-term):

- User B generates 100 signed prekeys
- Uploads to server
- Server stores them for future initiators

3. One-time Prekeys:

- User B generates additional one-time prekeys
- Used once and discarded
- Provides forward secrecy

4. Session Establishment:

- User A fetches User B's public identity key and a prekey from server
- User A generates ephemeral key pair
- Performs Diffie-Hellman key exchange
- Derives shared secret (session key)
- Encrypts message with session key
- Sends encrypted message + A's ephemeral public key

5. User B receives:

- Uses their private keys + A's ephemeral public key
- Derives the same session key
- Decrypts message
- Establishes bidirectional encrypted session

Message Encryption:

Each message is encrypted with:

$$\text{Encrypted Message} = \text{AES-256}(\text{message}, \text{session_key} + \text{message_counter})$$
$$\text{MAC} = \text{HMAC-SHA256}(\text{encrypted_message}, \text{mac_key})$$

The message_counter prevents replay attacks. Each message increments the counter.

Ratcheting (Forward Secrecy):

After each message, keys are “ratcheted” forward:

- New ephemeral key pair generated
- New session key derived
- Previous keys discarded

This means compromising today’s key doesn’t reveal:

- Yesterday’s messages,
- Or tomorrow’s messages.

Multi-Device Support:

User A has a phone and a laptop.

How does encryption work?

Approach 1: Sender Keys (Groups):

For groups, sending individual encrypted copies to each device is expensive.

Instead:

- User A generates a sender key for the group
- Encrypts sender key for each member using their session key
- Distributes sender key
- All future messages get encrypted with sender key
- Much more efficient for large groups

Approach 2: Session Per Device (1:1):

For 1:1 chats.

- Each device has its own identity
- User A's phone and laptop are separate entities
- When messaging User B, send two encrypted copies (one for each of B's devices)

Tradeoffs:

End-to-end encryption adds complexity.

- Can't search messages server-side
- Can't provide message previews in push notifications⁹ (would require server decryption)
- Server can't filter spam/abuse (content is opaque)
- Multi-device sync is harder

But the privacy benefits are worth it. Users trust WhatsApp because messages are truly private.

Ready for the best part?

Database Scaling: Sharding Strategy

With 10 billion messages per day, a single database doesn't cut it.

We need to split data across multiple database instances (sharding¹⁰).

Let's dive in!

Shard by message_id

Even distribution of data, but fetching all messages for a user requires querying multiple shards.

Your query hits every shard, aggregates results, then returns them.

So this is slow and expensive.

Shard by user_id

All messages for a user land on one shard. Perfect for our query patterns (give me all messages for this conversation).

The catch:

“Individual users can become hot spots¹¹.”

A celebrity with millions of messages creates an imbalanced shard. One shard is overloaded while others sit idle. Also groups should use a different strategy (single write, fan-out¹² on read).

Shard by timestamp

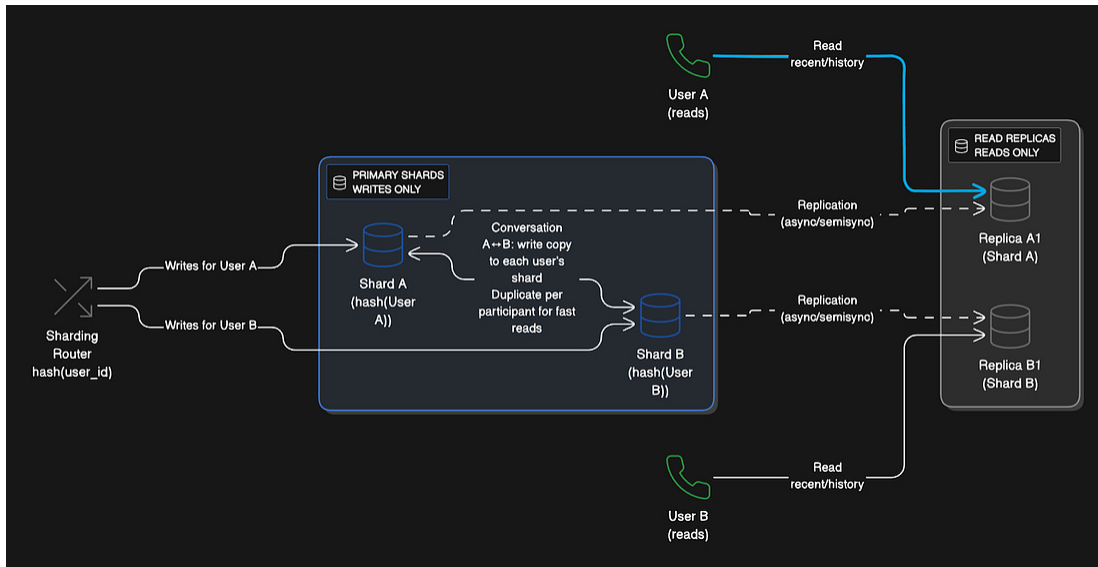
Good for archival access patterns (give me all messages from last month), but active users are spread across many shards... So your most common queries (recent messages for this user) still hit multiple shards.

Best approach - Shard by user_id with replication

Why we chose this:

It stores all messages where sender_id = X or receiver_id = X on shard determined by hash(X).

For a conversation between User A and User B, messages are written to both shards (once on A's shard, once on B's shard).



This doubles the write volume but makes reads blazing fast.

Each user queries only their own shard:

- No cross-shard queries.
- No aggregation overhead.
- Just a single shard lookup.

Add read replicas¹³ (copies of each shard that only handle read traffic) for each shard. Route read traffic to replicas. Only writes hit the primary¹⁴.

Tradeoffs:

We're writing each message twice (once per participant's shard), which doubles storage and write throughput requirements.

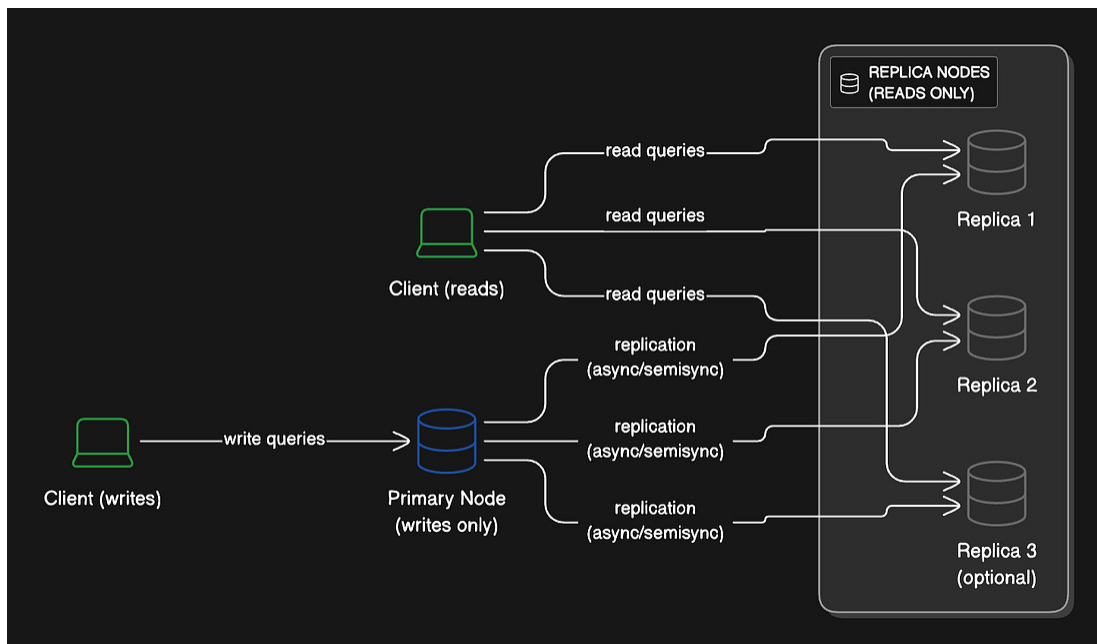
But reads are 10x more common than writes in messaging apps, so optimizing for read performance is the right call. The storage cost is worth the query speed we gain.

Keeping Data Available with Database Replication

We shard for write scalability. We replicate for read scalability and fault tolerance.

Let's dig into the replication strategy...

Replication Architecture:



For each shard:

- 1 Primary node (handles writes)
- 2-3 Replica nodes (handle reads)

Total: 100 shards × 3 nodes = 300 database nodes

Write Flow:

1. Write arrives at Primary

2. Primary writes to its local storage
3. Primary sends replication log to all Replicas:
 - Asynchronous: Don't wait for Replicas (faster writes)
 - Semi-synchronous: Wait for at least 1 Replica (safety)
 - Synchronous: Wait for all Replicas (slowest, highest safety)

We use semi-synchronous replication (Write operation):

1. Primary writes locally
2. Waits for ACK from 1 Replica (proves data is safe on 2 nodes)
3. Returns success to the client
4. Other Replicas catch up asynchronously

This balances write speed with safety.

If Primary dies, we have at least 1 Replica with the data.

Read Flow (Client query):

1. Client connects to load balancer¹⁵ for shard N
2. Load balancer routes to a random healthy Replica
3. Replica serves queries from local data
4. If Replica is lagging (replication delay), client might see stale data

Handling Replication Lag:

Replicas are typically 50-200ms behind Primary.

For most reads, this is fine. For critical reads (read-your-own-writes), we route to Primary:

- User A sends message → Writes to Primary
- User A immediately refreshes chat → Reads from Primary (not Replica)

This ensures User A sees their own message immediately.

- User B refreshes their chat → Reads from Replica

If Replica is slightly behind, User B sees the message 100ms later.

Not a problem!

Promotion on Failure:

When Primary dies, we need to promote a Replica:

Failure Detection:

1. Load balancer health checks¹⁶ fail (3 consecutive failures)
2. Load balancer marks Primary as unhealthy
3. Triggers failover process

Promotion:

1. Select the most up-to-date Replica (least lag)
2. Promote to Primary (update DNS or load balancer config)
3. Reconfigure other Replicas to sync from the new Primary
4. Takes 10-30 seconds total

During failover:

- Writes fail or queue (depends on client retry logic)
- Reads continue from Replicas
- After promotion, everything resumes normally

Split-Brain Prevention:

What if network partition makes two nodes think they're both Primary?

Solution:

Consensus via ZooKeeper or etcd

- Nodes use distributed consensus¹⁷ to elect Primary
- Only one node can hold the “Primary token”
- If a network partition occurs, only one side can get token
- Other side rejects writes (no split-brain¹⁸)

Cross-Datacenter Replication:

For disaster recovery, we replicate across data centers...

Datacenter A (Primary):

- Shard 1 Primary
- Shard 2 Primary

Datacenter B (Replica):

- Shard 1 Replica
- Shard 2 Replica

If Datacenter A burns down:

- Promote all Replicas in Datacenter B to Primary
- Traffic routes to Datacenter B
- System stays online (with higher latency for some users)

Tradeoffs:

Running 3x database nodes for replication is expensive (3x hardware costs).

But the benefits are massive:

■

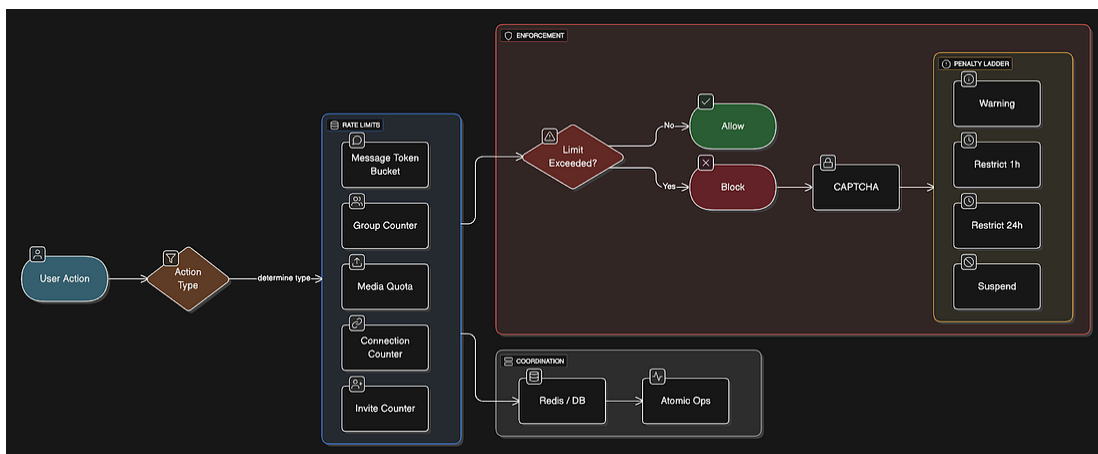
- 3x read capacity,
- fault tolerance,
- zero-downtime deployments (take down one node at a time).

Rate Limiting and Abuse Prevention

Without rate limits¹⁹, bad actors can spam, scrape data, or DDoS the system.

So let's build protection...

Types of Rate Limiting:



1. Per-User Message Limits:

Prevents spam and abuse.

Limits:

- 100 messages per minute per user
- 10 messages per second burst

- 5 groups created per hour
- 3 media uploads per minute

Implementation:

```
Token bucket algorithm

class RateLimiter:
    def __init__(self, capacity, refill_rate):
        self.capacity = capacity
        self.tokens = capacity
        self.refill_rate = refill_rate
        self.last_refill = time.now()

    def allow_request(self, user_id):
        self.refill()

        if self.tokens >= 1:
            self.tokens -= 1
            return True
        else:
            return False # Rate limited

    def refill(self):
        now = time.now()
        elapsed = now - self.last_refill
        tokens_to_add = elapsed * self.refill_rate
        self.tokens = min(self.capacity, self.tokens + tokens_to_add)
        self.last_refill = now
```

Stored in Redis:



When the user sends a message:

1. Check Redis for the current token count
2. If tokens are available, decrement and allow
3. If no tokens, return 429 Too Many Requests
4. Client shows: *"You're sending messages too quickly"*

2. Connection Rate Limiting:

Prevents connection flood attacks.

Limits:

- 5 connection attempts per minute per IP
- 10 connections per user across all devices

Implementation:

```
Connection rate limiting

# On WebSocket connection attempt
ip = request.remote_addr
attempts = redis.incr(f'conn_limit:ip:{ip}')
redis.expire(f'conn_limit:ip:{ip}', 60)

if attempts > 5:
    # Block IP temporarily
    redis.setex(f'blocked:ip:{ip}', 300, 1) # 5 min block
    return 429 # Too many connections
```

3. Group Invite Limits:

Prevents spam groups.

Limits:

- Can't be added to more than 5 groups per hour
- Can't create more than 3 groups per day

Check before adding a user to a group:

```
Group invite limits

invite_count = redis.incr(f'group_invites:user:{user_id}')
redis.expire(f'group_invites:user:{user_id}', 3600)

if invite_count > 5:
    return error("User cannot be added to more groups right now")
```

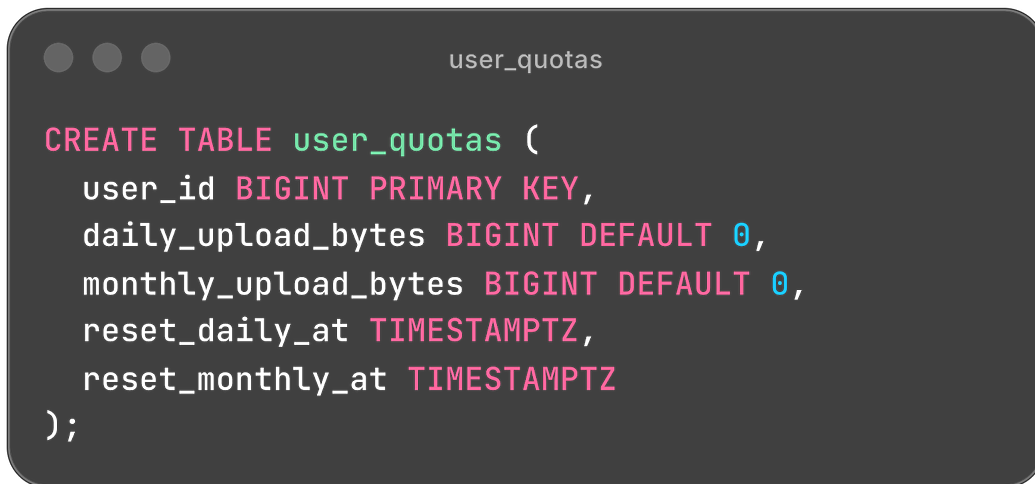
4. Media Upload Limits:

Prevents storage abuse.

Limits:

- 10MB per file
- 100MB per user per day
- 1GB per user per month

Track this in database:



```
CREATE TABLE user_quotas (  
  user_id BIGINT PRIMARY KEY,  
  daily_upload_bytes BIGINT DEFAULT 0,  
  monthly_upload_bytes BIGINT DEFAULT 0,  
  reset_daily_at TIMESTAMPTZ,  
  reset_monthly_at TIMESTAMPTZ  
);
```

CAPTCHA Challenges:

If the rate limit exceeds multiple times:

- Require CAPTCHA²⁰ before allowing more actions
- This prevents automated bots

Do Progressive Penalties:

- First violation: Warning
- Second violation: 1-hour restriction
- Third violation: 24-hour restriction
- Fourth violation: Account suspension

Distributed Rate Limiting:

With multiple chat servers, rate limits need to be coordinated.

Problem:

- User connects to Server 1, sends 50 messages.
- Then reconnects to Server 2, sends 50 more.

This bypasses the 100/min limit.

Solution:

Use **centralized rate limiting** in Redis.

- All servers check the same Redis key
 - Atomic operations (INCR) ensure accuracy
 - No way to bypass through server-hopping
-

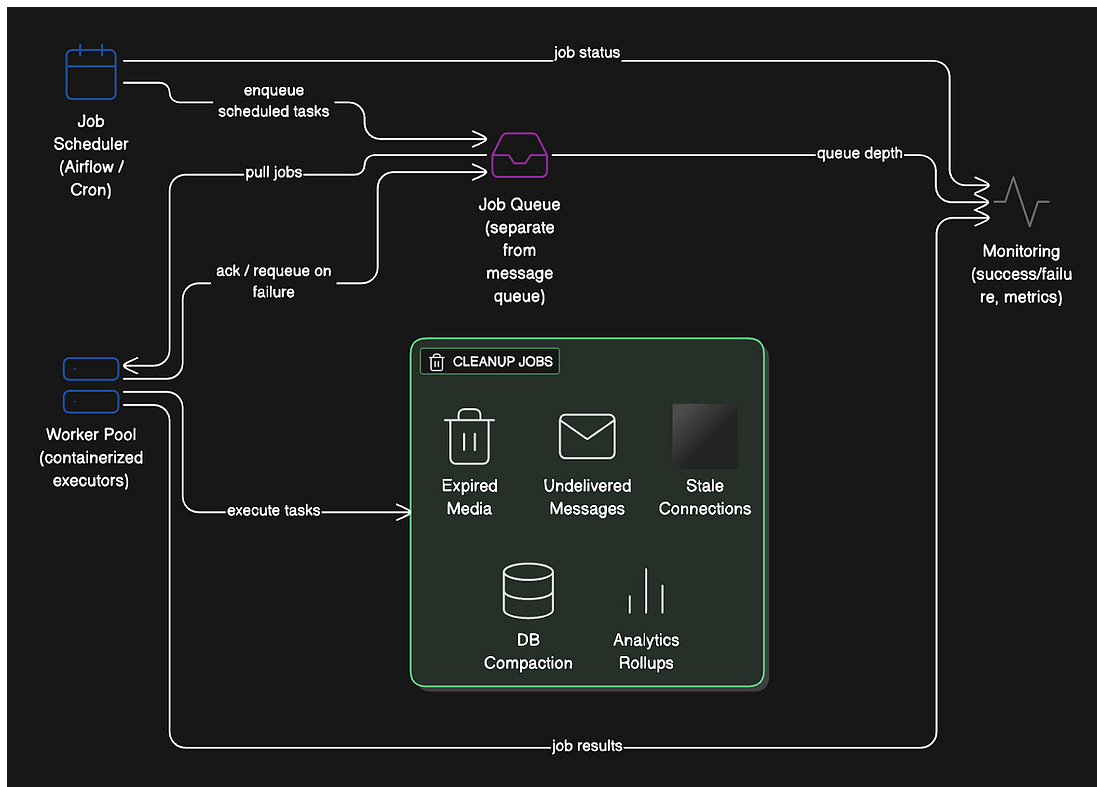
Scheduled Jobs: Cleanup Crew

Messaging systems accumulate garbage over time.

Expired media, old undelivered messages, stale connections.

So let's build the cleanup infrastructure.

Job Scheduler Architecture:



Components:

- Job Scheduler (Airflow or custom cron system)
- Worker Pool (containerized job executors)
- Job Queue (separate from message queue)
- Monitoring (track job success/failure)

Job 1: Expired Media Cleanup

Problem:

Users upload images, videos. Files stored in S3. After 30 days without being viewed, it should be deleted.

How do we track this?

Solution:

Media Metadata Table:

```
Media metadata

{
  media_id: "media_xyz789",
  upload_time: 1699189200000,
  last_accessed: 1699275600000,
  size_bytes: 2457600,
  message_id: "20241105120000000001",
  ttl_expires: 1701781200000 // 30 days from upload
}
```

Cleanup Job (runs daily at 2 AM):

1. Query: "SELECT * FROM media WHERE ttl_expires < NOW() LIMIT 10000"
2. For each expired media:
 - Delete from S3: DELETE object with media_id
 - Delete from CDN²¹ cache: PURGE cache entry
 - Update database: Mark as deleted
3. Process in batches (don't delete 1M files at once)
4. Sleep 1 second between batches (rate limiting)
5. Log deleted files for the audit trail

Optimization - Lazy Deletion:

Instead of actively deleting, set S3 lifecycle policies:

S3 Bucket Policy:

- Move files to Glacier after 30 days (cheap cold storage²²)

- Delete from Glacier after 90 days (permanent deletion)

This offloads deletion work to S3 itself. We just track metadata.

Job 2: Undelivered Message Cleanup

Problem:

Messages in Redis inboxes that are never delivered. User deleted their account, or the message is corrupted.

We don't want these sitting in Redis forever.

Solution:

Cleanup Job (runs every hour):

1. Scan all Redis inbox keys: `SCAN 0 MATCH inbox:* COUNT 1000`
2. For each inbox:
 - Check message age: `ZRANGE inbox:user123 0 0 WITHSCORES`
 - If oldest message > 30 days old:
 - Check if user still exists (query user database)
 - If user deleted: `DELETE` entire inbox
 - If user exists: Move old messages to the database archive
 - Maybe the user never opened the app, so we keep it for compliance.
3. Update metrics: Track cleaned inboxes and message count

Job 3: Database Compaction

Problem:

Cassandra stores deleted messages as tombstones²³.

And edited messages leave old versions. Over time, this could bloat disk space.

Solution:

Compaction Job (runs weekly during low-traffic hours).

1. For each shard:
 - Run Cassandra compaction: `nodetool compact messages`
 - This merges SSTables²⁴ and removes tombstones
 - Reclaims disk space
2. Monitor disk space before/after
3. Run on one shard at a time (don't hammer all shards simultaneously)

Job 4: Stale Connection Cleanup

Problem:

WebSocket connections that died but weren't cleaned up properly.
Ghost entries in Redis.

Solution:

Cleanup Job (runs every 5 minutes):

1. Scan connection registry: `SCAN 0 MATCH user:*`
2. For each entry:
 - Check `last_active` timestamp
 - If `last_active > 30 seconds ago`:
 - Verify connection with chat server (HTTP call to `/verify-connection`)
 - If server says *"not connected"*: DELETE Redis entry

- If server doesn't respond: Mark server as unhealthy

Job 5: Analytics Data Aggregation

Problem:

Raw message events in Kafka. Need daily/weekly/monthly rollups for analytics.

Solution:

Aggregation Job (runs daily at 3 AM).

1. Consume yesterday's events from Kafka
2. Calculate metrics:
 - Messages sent per user (histogram)
 - Active users per hour (time series)
 - Popular group sizes
 - Media upload volume
3. Write aggregated data to analytics database (e.g., ClickHouse)
4. Delete raw events from Kafka (retention policy)

Job Failure Handling:

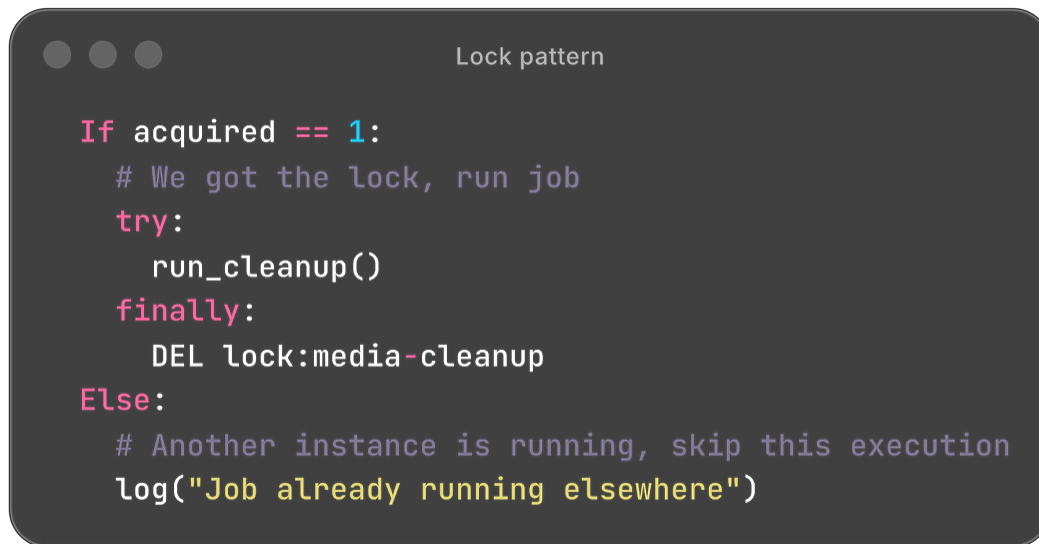
Each job.

1. Runs with timeout (kill if hangs for > 1 hour)
2. Retries 3 times on failure with exponential backoff
3. Sends an alert if all retries fail
4. Writes to job log: {job_id, start_time, end_time, status, error}
5. Prevents overlapping runs (use distributed lock²⁵ in Redis)

Distributed Lock Pattern:

Before the job starts:

```
acquired = SETNX lock:media-cleanup 1 EX 3600
```



```
Lock pattern

If acquired == 1:
    # We got the lock, run job
    try:
        run_cleanup()
    finally:
        DEL lock:media-cleanup
Else:
    # Another instance is running, skip this execution
    log("Job already running elsewhere")
```

This prevents two instances from running the same job simultaneously.

Tradeoffs:

Scheduled jobs add operational complexity (another thing to monitor and debug).

But the alternative is letting garbage accumulate until the system falls over. Regular cleanup is essential for long-term health.

Know someone who wants to learn system design? Consider gifting a subscription:

[Give a gift subscription](#)

What Makes This Design Production Ready

Message IDs That Don't Suck

Use timestamp-based IDs:

```
[timestamp_milliseconds][sequence][server_id]
```

Example: 20241105123456789001

Why we chose this approach:

- Sortable by creation time without extra columns
- No distributed ID generation needed (each server generates its own)
- Supports pagination (fetch messages before/after timestamp)
- Sequence number handles multiple messages in the same millisecond
- Server ID ensures global uniqueness even if clocks drift slightly

Tradeoffs:

Requires reasonably synchronized clocks across servers.

We use **NTP** (Network Time Protocol) to keep server clocks in sync. If clocks drift badly, messages might display slightly out of order, but the sequence number and server ID prevent actual collisions.

Message ID Alternatives

We chose timestamp-based IDs, but let's look at other options.

Option 1: Auto-Increment Database IDs

Just use PRIMARY KEY AUTO_INCREMENT in SQL.

Pros:

- Simple,
- Sequential,
- Easy to reason about.

Cons:

- Requires coordination (one database generates IDs)
- Bottleneck (all messages must go through one ID generator)
- Hard to shard (ID generation is centralized)

Doesn't work at our scale. We'd need distributed ID generation...

Option 2: UUID (Universally Unique Identifier)

Generate a random 128-bit UUID:

 550e8400-e29b-41d4-a716-446655440000

Pros:

- No coordination needed,
- Generate anywhere.

Cons:

- Not sortable by time
- Large size (36 characters as a string, 16 bytes as binary)
- Random, so no natural ordering for pagination

We'd need a separate timestamp field for sorting. This approach wastes space!

Option 3: Snowflake IDs (Twitter's approach)

64-bit ID structure

- 41 bits: Timestamp in milliseconds
- 10 bits: Machine ID (up to 1024 machines)
- 12 bits: Sequence number (4096 per millisecond per machine)

Example: 1699189200123456789

Pros:

- Sortable by time (timestamp is prefix)
- Compact (8 bytes, stores as BIGINT)
- No coordination needed (each machine generates independently)
- High throughput (4096 IDs per millisecond per machine)

Cons:

- Requires unique machine IDs (coordination at deploy time)
- Clock skew can cause issues (if clocks drift, ordering breaks)
- Limited to 69 years from epoch (41-bit timestamp)

This is a great option. WhatsApp likely uses something similar.

Option 4: ULID (Universally Unique Lexicographically Sortable Identifier)

128-bit ID structure:

- 48 bits: Timestamp in milliseconds
- 80 bits: Randomness

Example: 01ARZ3NDEKTSV4RRFFQ69G5FAV

Pros:

- Sortable by time
- No coordination needed
- Encodes to a short string (26 characters)

Cons:

- Larger than Snowflake (16 bytes vs 8 bytes)
- Randomness doesn't guarantee uniqueness (extremely unlikely collision though)

Good middle ground between UUID and Snowflake.

Our Choice: Timestamp + Sequence + Server ID

```
[timestamp_ms][sequence][server_id]
```

20241105 120000789 001 (server 1)

- 17 digits timestamp (milliseconds since epoch)
- 3 digits sequence (0-999)
- 3 digits server ID (0-999)
- Total: 23 digits (fits in BIGINT)

Pros:

- Human-readable (can see timestamp)
- Sortable
- No central coordination
- Compact

Cons:

- Limited to 999 servers (but we can extend)
- Limited to 1000 messages per millisecond per server

For messaging, being able to see the timestamp in the ID is useful for debugging.

And $1000 \text{ messages/ms/server} = 1\text{M messages/second per server}$, which is way more than we need.

Cache Failure: What Happens to Undelivered Messages?

Redis holds our undelivered message inboxes.

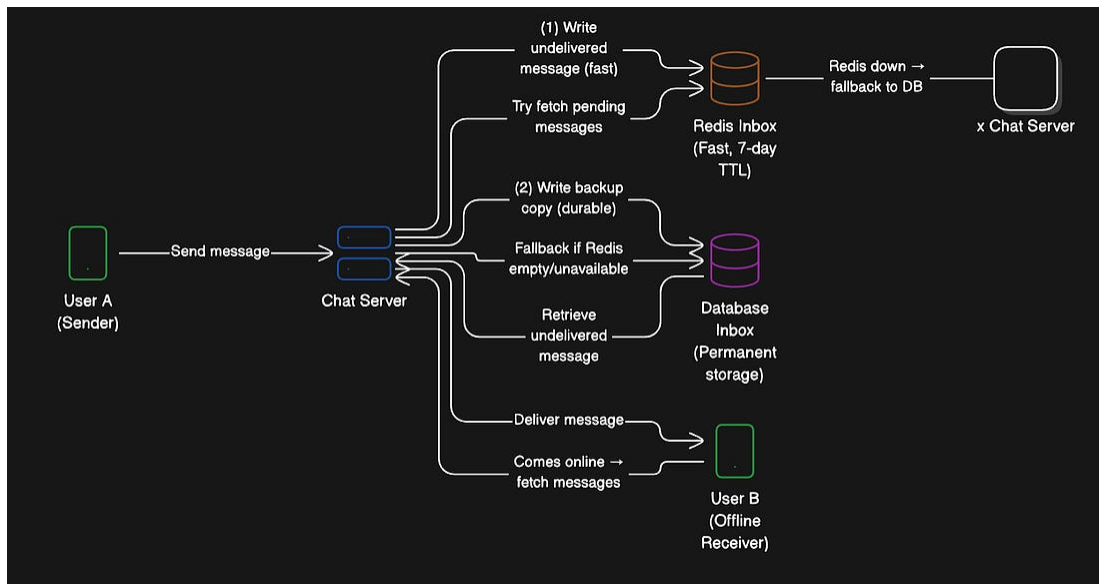
What if the Redis cluster goes down?

The Problem:

User A sends a message to an offline User B. Message goes to the Redis inbox.

Then Redis cluster crashes. So is the Message... gone?

Solution: Dual-Write Pattern



We don't write only to Redis. Instead, we write to two places...

When a message can't be delivered immediately:

1. Write to Redis inbox (fast, temporary storage)
2. Plus, write to the database inbox table (slower, permanent storage)

Redis inbox:

- Used for quick delivery when the user comes online
- TTL²⁶ of 7 days
- If Redis fails, we lose nothing critical

Database inbox:

- Permanent storage
- Used as a fallback if Redis is down
- Also serves as an audit log

Recovery Flow When Redis Fails:

Scenario: Redis cluster is down for 10 minutes

During outage:

1. Messages still get written to the database inbox
2. Users see *"message sent"* (because we ACK immediately)
3. Delivery just waits until the recipient comes online

When Redis recovers:

1. Background job runs: *"Sync inbox from database to Redis"*
2. Query: `SELECT * FROM message_inbox WHERE delivered_at IS NULL`
3. For each undelivered message:
 - Write to Redis inbox
 - Keep database record
4. Normal delivery resumes

When the user comes online (during or after the outage):

1. Client connects, server checks Redis inbox
2. If Redis is down or empty:
 - Fallback to database query
 - Load undelivered messages from database
 - Deliver to user
 - Mark as delivered in database

Write Path with Fallback:

```

Write with fallback

async function storeUndeliveredMessage(userId, message) {
  try {
    // Try Redis first (fast path)
    await redis.zadd(`inbox:${userId}`, message.timestamp, JSON.stringify(message));
  } catch (redisError) {
    // Redis is down, log it but don't fail
    logger.error('Redis unavailable, using database only', redisError);
  }

  // Always write to database (safety net)
  await db.query(
    'INSERT INTO message_inbox (user_id, message_id, sender_id, message_payload, created_at) VALUES ($1, $2, $3, $4, NOW())',
    [userId, message.id, message.senderId, message]
  );
}

```

Read Path with Fallback:

```

Read with fallback

async function getUndeliveredMessages(userId) {
  try {
    // Try Redis first (fast path)
    const messages = await redis.zrange(`inbox:${userId}`, 0, -1);
    if (messages.length > 0) {
      return messages.map(m => JSON.parse(m));
    }
  } catch (redisError) {
    logger.warn('Redis unavailable, falling back to database', redisError);
  }

  // Redis is down or empty, use database
  const result = await db.query(
    'SELECT message_payload FROM message_inbox WHERE user_id = $1 AND delivered_at IS NULL ORDER BY created_at',
    [userId]
  );
  return result.rows.map(r => r.message_payload);
}

```

Cleanup After Delivery:

```

Cleanup

async function markDelivered(userId, messageId) {
  // Remove from Redis
  try {
    await redis.zremrangebyscore(`inbox:${userId}`, messageId, messageId);
  } catch (redisError) {
    // Redis is down, not critical since we have database
    logger.warn('Could not remove from Redis inbox', redisError);
  }

  // Update database (source of truth)
  await db.query(
    'UPDATE message_inbox SET delivered_at = NOW() WHERE user_id = $1 AND message_id = $2',
    [userId, messageId]
  );
}

```

Why This Works:

Redis is an optimization, not the source of truth.

Database is the source of truth. If Redis fails:

- Messages don't get lost (they're in the database)
- Delivery just gets slower (database queries instead of Redis)
- System stays functional (degraded but operational)

Periodic Sync Job:

Background job (runs every 5 minutes).

1. Query database: *"Give me all undelivered messages"*
2. For each message:
 - Check if it's in Redis
 - If not, add it to Redis
3. This handles cases where Redis lost data (memory eviction, failure)

Tradeoffs:

Dual writes add latency (writing to the database is slower than Redis).

But we write to the database asynchronously (don't block the response). The safety is worth the slight complexity.

Analytics and Monitoring: Observability at Scale

You can't run a billion-user system blind.

We need comprehensive monitoring²⁷ and analytics...

Architecture:

- Application logs (structured JSON)
- Metrics ([Prometheus/Grafana](#))
- Distributed traces ([Jaeger](#))
- Error tracking (Sentry)

Critical Metrics:

Chat Servers:

- Active connections (alert >80% capacity),
- Message throughput,
- WebSocket errors (alert >1%),
- Memory/CPU usage

Message Delivery:

- End-to-end latency (p99 <500ms),
- Delivery success rate (>99.9%),
- Undelivered backlog,

- Queue consumer lag

Database:

- Query latency (p99: 100ms reads, 500ms writes),
- Replication lag²⁸ (<5s),
- Connection pool saturation,
- Disk usage

Redis:

- Memory usage,
- Eviction rate (should be 0),
- Commands/sec,
- Pub/Sub latency

Business Metrics:

- DAU/MAU,
- Messages per user,
- Group participation,
- Media upload success rate

Distributed Tracing:

Every request gets a trace_id²⁹:

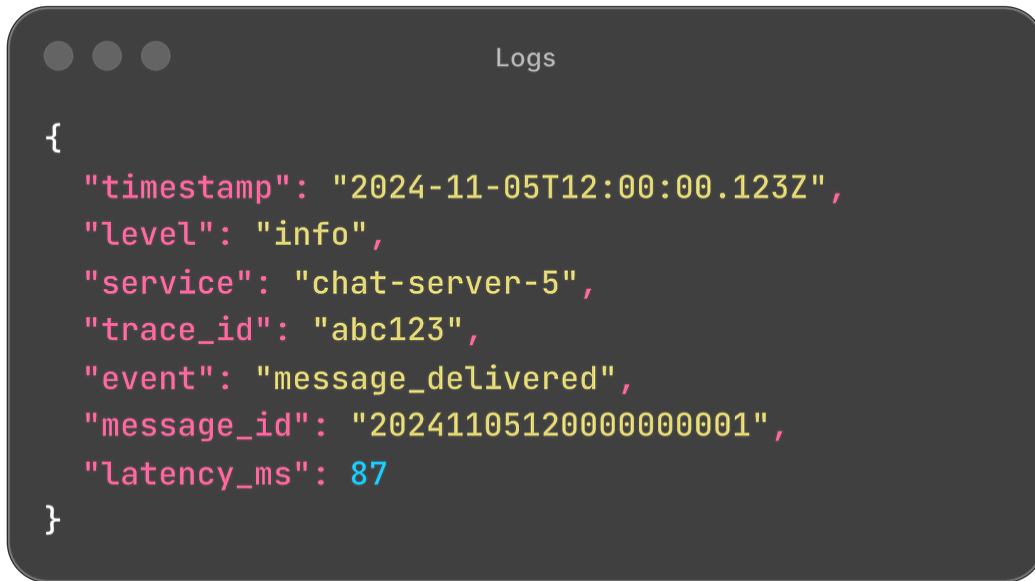
User A sends message (trace_id: abc123):

- [abc123] Chat Server receives (2ms)
- [abc123] Check Redis location (5ms)
- [abc123] Route to Server 2 (15ms)
- [abc123] Deliver via WebSocket (8ms)
- [abc123] Write to Queue (12ms)

```
- [abc123] Database write (45ms)
Total: 87ms
```

If something is slow, you can see exactly which component is causing it.

Structured Logging:



```
{
  "timestamp": "2024-11-05T12:00:00.123Z",
  "level": "info",
  "service": "chat-server-5",
  "trace_id": "abc123",
  "event": "message_delivered",
  "message_id": "20241105120000000001",
  "latency_ms": 87
}
```

Query examples:

- Show failed deliveries last hour:

```
level:error AND event:message_delivery_failed
```

- What's p99 latency³⁰ for user 456?:

```
user_id:456 | stats percentile(latency_ms, 99)
```

On-Call Runbooks:

When alerts fire, engineers need action steps:

Alert: "Redis memory > 85%"

1. Check the eviction rate in the dashboard
2. If evicting: Scale cluster or clear old data
3. If not evicting: Check for memory leak (key count trends)
4. If leak: Identify service, restart, file bug

Tradeoffs:

Monitoring adds overhead (CPU/memory/network). Yet running blind is the worst.

Know someone who wants to learn system design? Consider gifting a subscription:

[Give a gift subscription](#)

Multi-Region and Global Distribution

WhatsApp serves users globally.

A single data center in Virginia can't serve users in Tokyo with low latency. Time to go multi-region.

Regions:

- US East (Virginia)
- US West (Oregon)

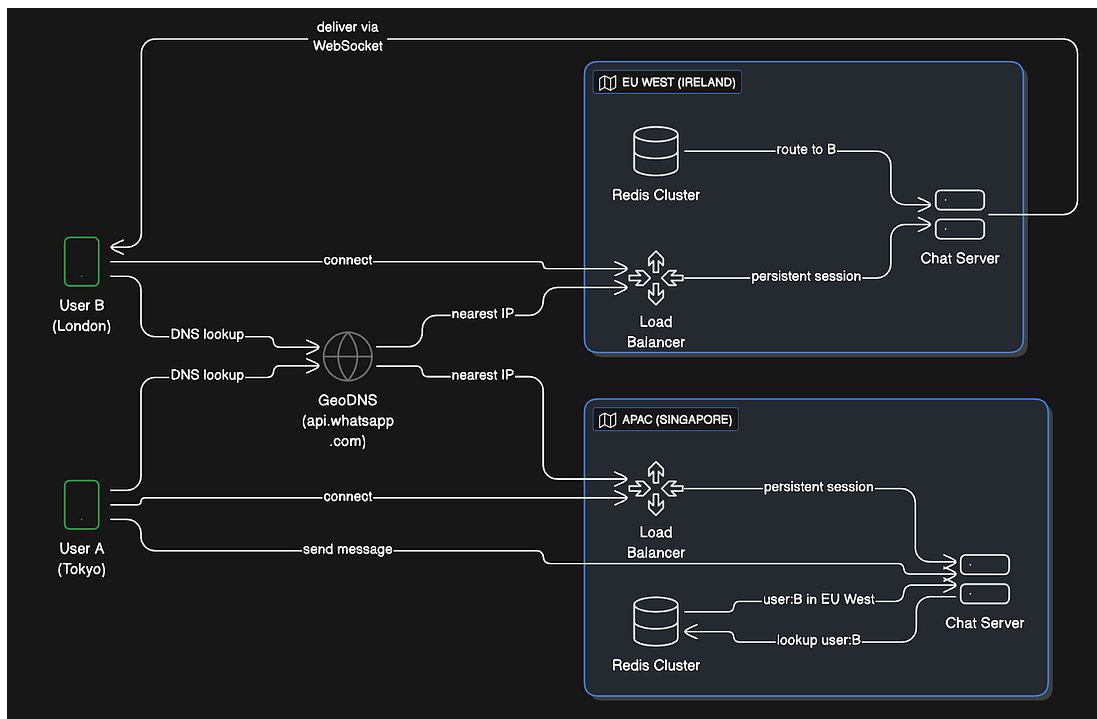
- EU West (Ireland)
- Asia Pacific (Singapore)
- Middle East (Bahrain)
- South America (São Paulo)

Each region has a full stack:

- Chat servers
- Redis clusters
- Database shards
- Message queues

Regional Routing:

Users connect to the nearest region for low latency:



User in Tokyo opens app:

1. App does a DNS lookup for `api.whatsapp.com`
2. GeoDNS³¹ returns the IP of the Asia Pacific load balancer
3. User connects to the Singapore region
4. All their traffic stays in Singapore (low latency)

Cross-Region Messages:

User A in Tokyo messages User B in London. So what happens then?

1. User A sends a message to the Singapore chat server
2. Singapore server checks Redis: *"Where is User B?"*
3. Redis returns: *"User B is on EU West chat server."*
4. Singapore server publishes to global Redis Pub/Sub: Channel:
`region:eu-west:user:456`
5. EU West Redis receives, routes to User B's chat server
6. User B receives the message

Message flow:

 Tokyo → Singapore server → Global Redis → Ireland Redis → London

Total latency: ~150-200ms (acceptable for messaging).

Database Sharding Across Regions:

Option 1: Users pinned to home region

- User A is registered in Tokyo, but their data is stored in Singapore.
- Even if User A travels to London, their data stays in Singapore.
- Reads/writes always hit Singapore (higher latency when traveling).

Pros:

- Simple,

- Data stays in one place.

Cons:

- Poor experience when traveling

Option 2: Replicate user data to all regions

- User A's data is in all regions.
- Writes go to home region, replicated to others.
- So Reads are always local (fast).

Pros:

- Fast everywhere

Cons:

- Storage cost (5x if 5 regions),
- Consistency challenges.

We use a hybrid approach:

- User metadata (profile, contacts): Replicated globally.
- Message history: Stored in home region only.
- Active conversations: Cached in the current region

When User A travels from Tokyo to London:

1. Profile data already in London (replicated)
2. Active chats cached on first load
3. Old message history fetched from Singapore on demand

Consistency Challenges:

User A has devices in Tokyo and London...

They send a message from the Tokyo device. London device should see it immediately.

Solution

Global Pub/Sub + Regional Pub/Sub.

Message sent from Tokyo:

1. Written to Singapore database (home region)
2. Published to Singapore Redis (for local devices)
3. Also published to Global Redis (for cross-region devices)
4. Global Redis fans out to all regions
5. London Redis receives and delivers to User A's London device

This adds ~50-100ms latency for cross-region delivery, but keeps everything in sync.

Conflict Resolution:

User A edits a message on their Tokyo device. Simultaneously edits the same message on the London device.

Which wins?

| Last-Write-Wins with Timestamps:

- Edit from Tokyo: timestamp = 1699189200123
- Edit from London: timestamp = 1699189200456

London edit wins (higher timestamp). Both devices eventually converge at London's version.

If timestamps are identical (rare):

- Use `device_id` as a tiebreaker (lexicographically)

- Consistent across all regions

Failover Across Regions:

If Singapore region goes down, redirect Tokyo users to the backup region:

1. Health monitoring detects Singapore outage
2. GeoDNS updated: Tokyo users now route to Hong Kong (nearest healthy region)
3. Users reconnect (automatic retry)
4. Hong Kong servers serve Tokyo users (slightly higher latency)
5. Data will eventually be replicated when Singapore recovers

Regulations and Data Residency³²:

Some countries require data to remain within their borders (e.g., EU GDPR, China, Russia).

Compliance Strategy:

EU.

- Data must stay in the EU region
- Cannot be replicated to the US
- Special region configuration

China.

- Separate deployment (different regulations)
- No data leaving China
- Different legal entity

Rest of the world.

- Data can be replicated across regions

Tradeoffs:

Multi-region deployment massively increases complexity (5x infrastructure, cross-region networking, consistency challenges) and costs.

But it's necessary on a global scale. Users demand low latency regardless of location.

Handling Failures Gracefully

Chat server dies:

- All connected clients lose WebSocket connections
- Clients automatically retry with exponential back-off (wait time doubles after each failed attempt to prevent overwhelming the system)
- Load balancer health checks detect a dead server within seconds
- New connections route to healthy servers
- Clients query inbox for missed messages during downtime

Redis cluster node fails:

- Clients can't receive real-time messages temporarily
- Pub/Sub messages lost (by design, since we have durability elsewhere)
- No problem! Messages are still in the Message Queue
- When clients reconnect, they fetch from the inbox
- Redis cluster rebalances automatically, and recovery is transparent

Database replica lags:

- Writes always go to the primary
- Reads might see slightly stale data
- For messaging, eventual consistency³³ is fine
- If you absolutely need your own write immediately, route that read to the primary

Message queue backlog builds up:

- Chat servers keep acknowledging receives
- Users see “*sent*” status immediately
- Message Storage Service scales horizontally (add more consumer instances) to consume faster
- In extreme cases, add more consumers
- Queue persists messages, so there’s no data loss even if consumers are slow

Message Queue Failure and Recovery

Kafka is our message queue.

What if it fails?

Kafka Architecture Recap:

- Multiple brokers (servers) in a cluster
- Topics are split into partitions
- Each partition has a leader and replicas
- Producers write to the leader
- Consumers read from the leader
- Replicas provide redundancy

Failure Scenario 1: Broker Dies

- Kafka cluster: 5 brokers
- Broker 3 dies (hardware failure)

Impact:

- Partitions where Broker 3 was the leader: Leadership election
- Kafka automatically promotes a replica to the leader (within seconds)
- Producers and consumers reconnect with the new leader
- No message loss (replicas have the data)

Recovery:

- Replace Broker 3 (new hardware)
- Broker rejoins cluster
- Kafka rebalances partitions over time

Failure Scenario 2: Entire Kafka Cluster Down

This is catastrophic but rare (requires a data center failure or major misconfiguration).

Immediate Impact:

- Chat servers can't publish messages to the queue
- Message Storage Service can't consume messages
- Messages not persisted to the database

Mitigation - Local Buffer:

```
Chat server maintains local buffer

const messageBuffer = [];

async function publishMessage(message) {
  try {
    await kafka.publish('messages', message);
  } catch (kafkaError) {
    // Kafka is down, buffer locally
    messageBuffer.push(message);
    logger.error('Kafka unavailable, buffering message', {messageId: message.id});

    // Alert operations team
    alertOncall('Kafka cluster unreachable');

    // If buffer gets too large (> 100k messages), start dropping oldest
    if (messageBuffer.length > 100000) {
      const dropped = messageBuffer.shift();
      logger.error('Buffer full, dropping message', {messageId: dropped.id});
    }
  }
}

// Background job: Flush buffer when Kafka recovers
setInterval(async () => {
  if (messageBuffer.length > 0 && kafkaHealthy()) {
    const batch = messageBuffer.splice(0, 1000);
    for (const msg of batch) {
      await kafka.publish('messages', msg);
    }
  }
}, 5000);
```

Why This Works:

- Messages still acknowledged by users (they see “sent”)
- Messages delivered to recipients in real-time (via Redis Pub/Sub)
- Database persistence is delayed but not lost
- When Kafka recovers, buffer flushes

Permanent Loss Prevention:

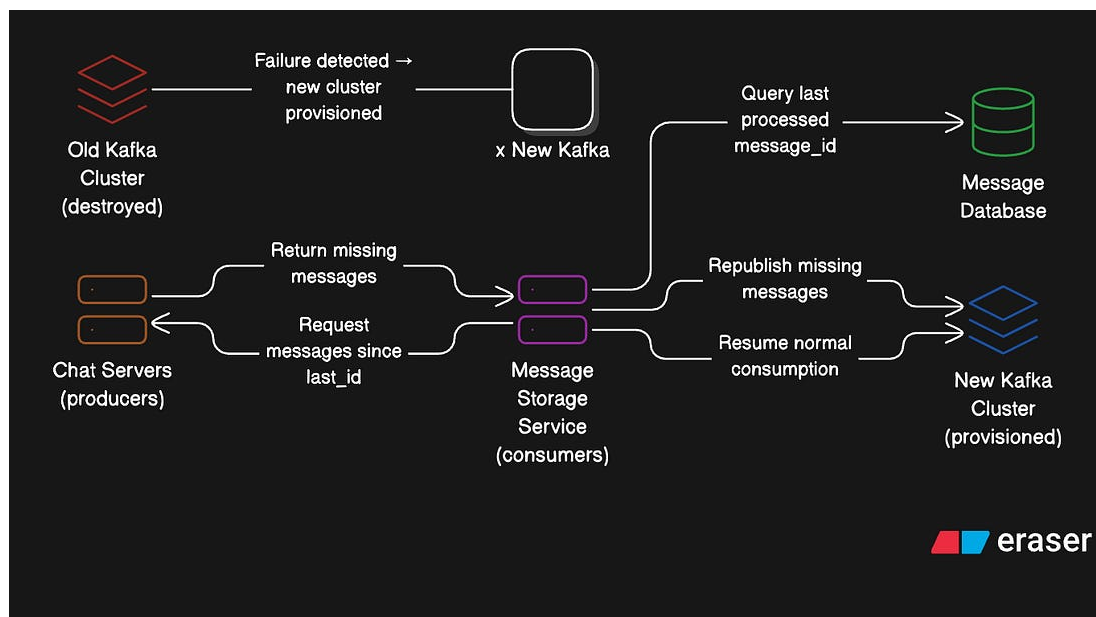
If a chat server crashes while holding buffered messages, those messages are lost.

To prevent this, we need **dual persistence**...

When publishing to Kafka fails:

1. Buffer message in memory (for quick retry)
2. Also write to local disk (append-only log file)
3. When Kafka recovers:
 - Flush memory buffer
 - Replay from disk log file
 - Delete the disk log after successful publish

Disaster Recovery - Kafka Cluster Restoration:



If the Kafka cluster is completely destroyed:

1. Provision a new Kafka cluster
2. Message Storage Service has a gap in processing
3. Query database: "What was the last message_id we processed?"
4. Query chat servers: "What messages did you send since then?"

5. Republish missing messages to the new Kafka cluster
6. Resume normal operation

This is manual and takes hours, but it prevents data loss.

Monitoring and Alerting:

Critical Kafka metrics:

- Under-replicated partitions (alert immediately)
- Broker offline (alert immediately)
- Consumer lag > 10k messages (alert within 5 minutes)
- Disk usage > 80% (alert before it fills)

Kafka health check:

- Producers publish test message every 10 seconds
- Consumers verify receipt
- If test message not received within 30 seconds, trigger an alert

Tradeoffs:

Buffering on chat servers adds memory pressure and complexity.

But it prevents message loss during Kafka outages. The alternative (blocking message sends until Kafka recovers) creates a terrible user experience.

Thundering Herd Problem

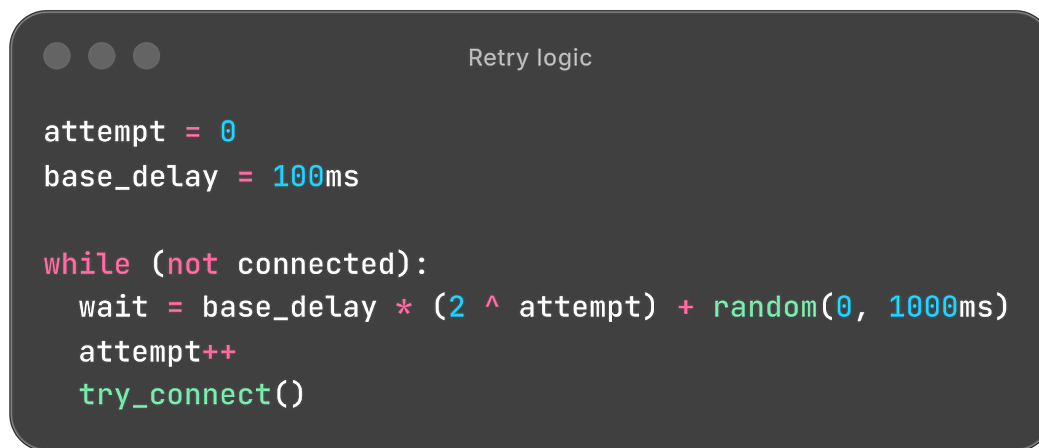
When a chat server crashes, thousands of clients try to reconnect simultaneously.

This can overwhelm the remaining servers³⁴.

Solution: Exponential backoff with jitter

Exponential backoff means each retry waits longer than the previous one. Jitter³⁵ adds randomness to spread out the retries.

- Client retry logic: First retry: 100ms
- Second retry: 200-1200ms (jitter spreads out clients)
- Third retry: 400-1400ms
- Max delay: cap at 30 seconds



```
attempt = 0
base_delay = 100ms

while (not connected):
    wait = base_delay * (2 ^ attempt) + random(0, 1000ms)
    attempt++
    try_connect()
```

Data Schema Versioning and Backward Compatibility

As the app evolves, we add features.

So how do we change schemas without breaking old clients?

The Problem:

Old app version expects:

```
{
  "action": "new_message",
  "message_id": "...",
  "content": "..."
}
```

New version adds reactions:

```
{
  "action": "namesake",
  "message_id": "...",
  "content": "...",
  "reactions": [...]
}
```

Next let's think about what happens when the old client receives the new format:

Solution: Additive Changes Only

Rule 1: Never remove or rename fields. Instead, only add new ones.

Old clients ignore unknown fields and only parse what they know:

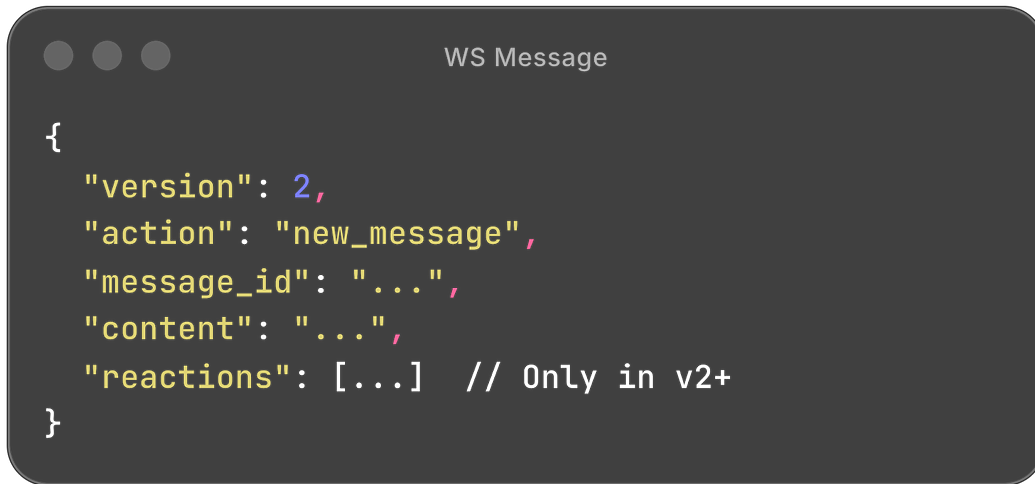


```
Old client code

const message = JSON.parse(data);
console.log(message.content); // Works fine, ignores "reactions"
```

Protocol Versioning in Messages:

Every WebSocket message includes a version.

A dark-themed window titled "WS Message" with three gray window control buttons (minimize, maximize, close) in the top-left corner. The window contains a JSON object with the following fields: "version" (value 2), "action" (value "new_message"), "message_id" (value "..."), "content" (value "..."), and "reactions" (value [...]). A comment "// Only in v2+" is placed after the "reactions" field.

```
{
  "version": 2,
  "action": "new_message",
  "message_id": "...",
  "content": "...",
  "reactions": [...] // Only in v2+
}
```

Server checks client version on connect:

A dark-themed window titled "Client version" with three gray window control buttons (minimize, maximize, close) in the top-left corner. The window contains a JSON object with the following fields: "action" (value "connect"), "client_version" (value "2.24.5"), and "protocol_version" (value 2).

```
{
  "action": "connect",
  "client_version": "2.24.5",
  "protocol_version": 2
}
```

If the client is v1, the server strips v2 fields before sending.

Server-Side Compatibility Layer:

```
Server side compatibility

function formatMessageForClient(message, clientVersion) {
  if (clientVersion < 2) {
    // Old client, remove new fields
    const {reactions, reply_chain, ...compatible} = message;
    return compatible;
  }
  // New client, send everything
  return message;
}
```

Database Schema Evolution:

When adding a column to the messages table.

```
Schema evolution

-- Step 1: Add column as nullable
ALTER TABLE messages ADD COLUMN reactions JSONB DEFAULT NULL;

-- Step 2: Deploy new code that writes to this column
-- (old code ignores it, reads still work)

-- Step 3: Backfill existing messages (if needed)
UPDATE messages SET reactions = '[]' WHERE reactions IS NULL;

-- Step 4: Make NOT NULL (only after all clients upgraded)
ALTER TABLE messages ALTER COLUMN reactions SET NOT NULL;
```

Handling Breaking Changes:

Sometimes you must break compatibility (rare but it happens):

Example: Changing message_id format

- Old format: Sequential integers (1, 2, 3...)
- New format: Timestamp-based (20241105123456789001)

Here's a migration³⁶ strategy that we can use in this case:

1. Add a new field ``message_id_v2`` alongside old ``message_id``
2. Write to both fields for 6 months
3. Clients gradually update to use ``message_id_v2``
4. After 95% adoption, deprecate ``message_id``
5. After 99% adoption, remove ``message_id``

API Versioning Strategy:

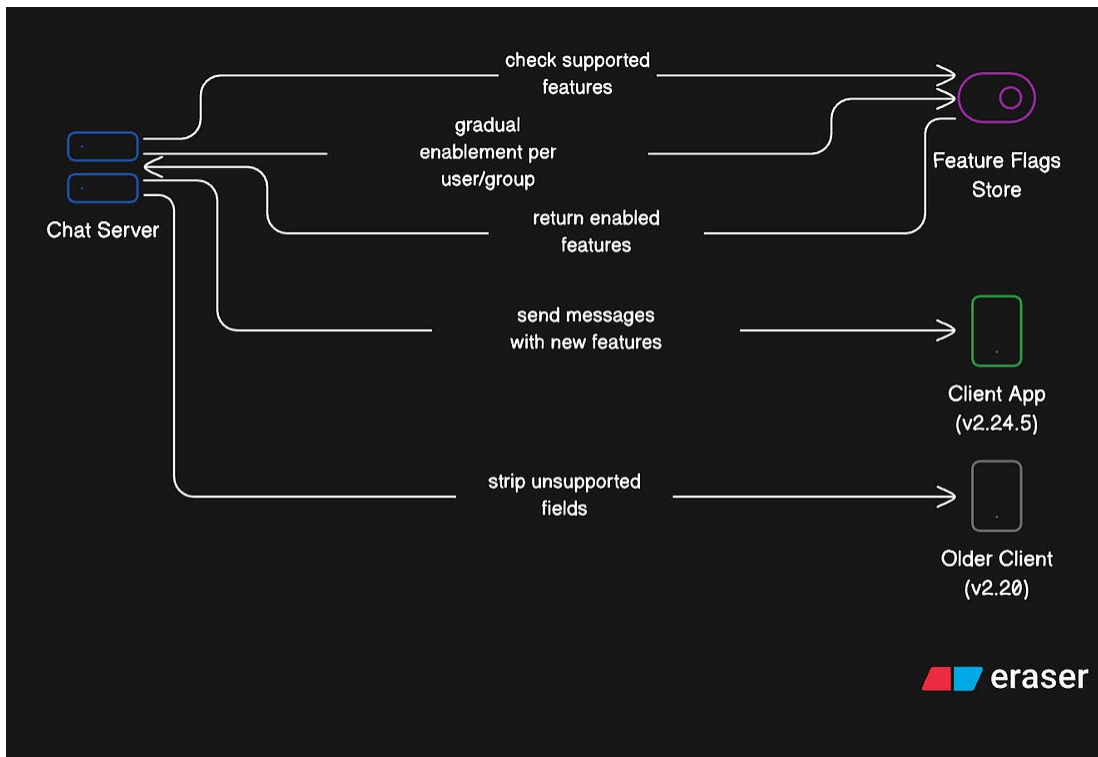
For this system,

- We use protocol versions (`wss://chat.whatsapp.com/ws`),
- Not URL versioning (`wss://chat.whatsapp.com/v1/ws`).

The client specifies the version in the initial handshake. Server adapts responses based on that version.

Feature Flags:

For experimental features, we can use feature flags³⁷.



```
Feature flags

{
  "action": "connect",
  "client_version": "2.24.5",
  "features": ["reactions", "polls", "communities"]
}
```

Server only sends the features the client supports. Gradual rollout without breaking old clients.

Database Index Changes:

Adding an index is safe (doesn't affect reads/writes):

```
CREATE INDEX CONCURRENTLY idx_messages_timestamp ON  
messages(timestamp);
```

Removing an index requires care:

1. Deploy code that doesn't use the index
2. Wait 1 week (ensure no old code running)
3. Drop the index

NoSQL Schema Evolution:

Cassandra is flexible³⁸ but still needs care:

1. Add new column (easy, no migration needed)

```
ALTER TABLE messages ADD reply_to TEXT;
```

2. Change column type (hard, requires full migration)

Can't change type directly, must:

- 1. Add new column with new type
- 2. Backfill data
- 3. Update code to use new column
- 4. Drop old column (after transition period)

Cracking the System Design Interview

You've seen the complete architecture.

Now let's talk about how to actually nail these types of interviews:

Start Simple, Then Scale

Don't jump straight into Redis Pub/Sub and database sharding.

Begin with a single chat server handling WebSocket connections. Show you can build something that works for 1000 users first. Then identify the bottleneck (single server can't handle a billion users) and explain how you'd scale horizontally.

Interviewers want to see your thought process, not a memorized solution.

Understand the Tradeoffs

Every decision has costs.

WebSockets use more memory than HTTP but provide real-time delivery. Sharding by `user_id` creates hot spots but makes reads blazing fast. End-to-end encryption protects privacy but prevents server-side spam filtering.

When you propose a solution, always mention what you're giving up to get it.

This shows you're thinking like a senior engineer, not just listing technologies.

Know Your Numbers

Be ready to do back-of-the-envelope calculations.

500 million daily active users sending 20 messages each is 10 billion messages per day. That's 115,000 messages per second average, but 350,000-500,000 at peak.

Can your database handle that write throughput?

How much storage do you need?

Interviewers use these numbers to push you toward the right design decisions.

If you calculate messages are 1KB each, and you're storing 10 TB per day, you'll naturally arrive at the need for distributed storage.

Think About Failures

The best candidates don't just design for the happy path.

What happens when the Redis cluster goes down?

When a chat server crashes with 100,000 connected users?

When Kafka backlogs by a million messages?

Show you've thought about failure modes and recovery strategies. Dual writes to Redis and the database. Client-side retry logic with exponential backoff. Circuit breakers³⁹ and fallback mechanisms.

This is what separates good designs from production-ready ones.

Communication Matters More Than Perfection

You won't design a perfect system in 1-2 hours.

That's not the goal. The interviewer wants to see how you think, how you handle ambiguity, and whether you can explain complex ideas clearly.

Ask clarifying questions:

"Should we optimize for read or write throughput?"

“Do we need strong consistency or is eventual consistency acceptable?”

“What’s more important: low latency or guaranteed delivery?”

Talk through your reasoning out loud. When you hit a problem, explain what you’re considering. The interviewer might guide you if you’re stuck, but only if they understand where you’re going.

What Actually Matters

You don’t need to memorize every detail in this article. But you should understand:

- Why WebSockets beat polling for real-time messaging
- How to route messages between users on different servers (Redis Pub/Sub)
- Why databases need sharding and replication at scale
- How to handle offline message delivery with inboxes
- The challenges of multi-device sync and end-to-end encryption

Get these core concepts right, show good judgment on tradeoffs, and communicate clearly. That’s how you pass the interview.

The system we built here handles billions of users, but it started with simple ideas scaled thoughtfully. That’s exactly the approach you should take.

Good luck with your interviews!



I’d like to thank [Hayk](#) for writing this newsletter!

- Plus, don’t forget to join his [**YouTube channel**](#).

It'll help you master the essential system design skills, not just in theory but in practice, and land senior developer roles.

If you find this newsletter valuable, share it with a friend. There are [group discounts](#), [gift options](#), and [referral rewards](#) available.

[Give a gift subscription](#)



👉 Find me on [LinkedIn](#) | [Twitter](#) | [Threads](#) | [Instagram](#)

- 1 **End-to-end encryption** means messages are encrypted on the sender's device and only decrypted on the recipient's device, so servers can't read content.
- 2 **Consistent hashing** is a distribution technique that maps keys to servers so that data movement is minimized when servers are added or removed.
- 3 **Pub/Sub (Publish-Subscribe)** is a messaging pattern where publishers send messages to channels, and subscribers receive messages from channels they're interested in.
- 4 **WebSocket** is a protocol that establishes a persistent, two-way connection between a client and a server, allowing both to send data at any time without repeated handshakes.

- 5 **Message Queue** is a system that temporarily stores messages in order, decoupling the sender from the receiver so they don't need to process messages at the same time.
- 6 **Exponential backoff** is a retry strategy where wait times double after each failure (100ms, 200ms, 400ms, 800ms) to avoid overwhelming systems.
- 7 **Dead letter queue** is a special queue for messages that failed processing multiple times, allowing later investigation without blocking other messages.
- 8 **Forward secrecy** means that compromising today's encryption key doesn't allow past messages to be decrypted, because old keys are deleted.
- 9 **Push notifications** are messages sent from a server to a mobile device that appear even when the app isn't open, delivered through APNs (iOS) or FCM (Android).
- 10 **Sharding** is splitting a database into multiple smaller pieces (shards) across different servers, where each shard holds a subset of the data.
- 11 **Hot shard** is a database shard receiving disproportionately high traffic compared to others, creating a performance bottleneck.
- 12 **Fan-out** is the delivery of a single message to multiple recipients, where the workload scales linearly with the recipient count (1000 recipients = 1000 deliveries).
- 13 **Read replica** is a copy of a database that only handles read requests, allowing the primary database to focus on writes while replicas scale read capacity.
- 14 **Primary (leader) node** is the main database that handles all write operations, while replica (follower) nodes copy its data and handle read operations.
- 15 **Layer 4 load balancing** routes traffic based on IP address and port (network layer), while Layer 7 load balancing routes traffic based on HTTP content (application layer).
- 16 **Health check** is a periodic request to a service endpoint to verify if it's functioning properly, used by load balancers to route traffic only to healthy

servers.

- 17 **Consensus protocol** is an algorithm (such as Raft or Paxos) that ensures multiple servers agree on a single value even when some servers fail.
- 18 **Split-brain** occurs when a network partition causes multiple nodes to think they're the primary, potentially leading to data conflicts.
- 19 **Rate limiting** restricts the number of requests a user can make in a time period (e.g., 100 messages per minute) to prevent spam and abuse.
- 20 **CAPTCHA** is a challenge-response test to verify that the user is human, typically requiring solving a puzzle or identifying images.
- 21 **Content Delivery Network (CDN)** is a network of servers distributed globally that cache content and serve it from locations closest to users.
- 22 **Cold storage** is cheaper, slower cloud storage for rarely accessed data (like AWS Glacier), versus hot storage for frequently accessed data.
- 23 **Tombstone** is a marker indicating deleted data in databases that use append-only logs, preventing it from reappearing.
- 24 **SSTable** (Sorted String Table) is an immutable file on disk that stores data in sorted order so databases like Cassandra can read and write data efficiently at a large scale.
- 25 **Distributed lock** is a mechanism that ensures only one process across multiple servers can execute a critical operation at a time.
- 26 **Time To Live (TTL)** is the expiration time after which data is automatically deleted when stored in caches or queues.
- 27 **Observability** is the ability to understand a system's internal state by examining its outputs (logs, metrics, traces).
- 28 **Replication lag** is the delay between when data is written to the primary database and when it appears on replica databases.

- 29 **Distributed tracing** tracks a request's journey across multiple services, showing exactly where time is spent and where failures occur.
- 30 **Percentiles (p99, p95)** are statistical measures: p99 means that 99% of requests are faster than this value, showing worst-case performance.
- 31 **GeoDNS** is a DNS service that returns different IP addresses based on a user's geographic location, routing traffic to the nearest datacenter.
- 32 **Data residency** is the legal requirement that certain data must be stored within specific geographic boundaries (e.g., EU data must be stored within the EU).
- 33 **Eventual consistency** means data will become consistent across all nodes eventually, but may be temporarily out of sync during replication.
- 34 **A thundering herd** occurs when many clients simultaneously retry after a failure, overwhelming the system with a sudden spike in requests.
- 35 **Jitter** is a random variation added to retry timing to prevent many clients from retrying simultaneously and overwhelming the system.
- 36 **Schema migration** is the process of changing the database structure (adding columns, changing types) while keeping the system running.
- 37 **Feature flag** is a configuration toggle that enables/disables features for specific users, allowing gradual rollouts and A/B testing.
- 38 **NoSQL database** is designed for high write throughput and flexible schemas, unlike traditional SQL databases with fixed table structures.
- 39 **Circuit breaker** is a pattern that stops requests to a failing service after repeated errors and only resumes after a cooldown period.